Programación Web

El nivel de lógica del negocio

Dr. Alejandro Zunino & Dr. Alfredo Teyseyre
alejandro.zunino@isistan.unicen.edu.ar, alfredo.teyseyre@isistan.unicen.edu.ar









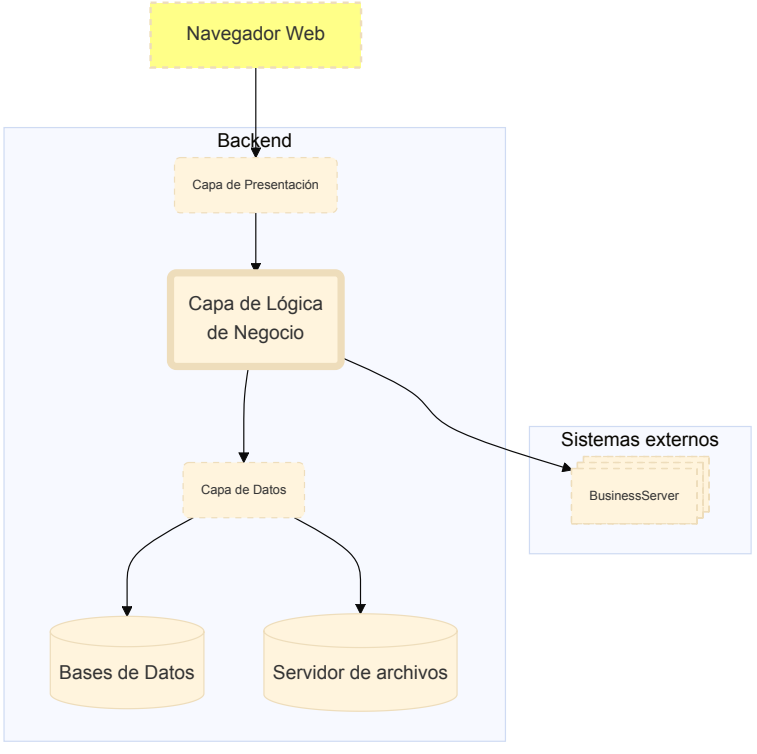
Contenidos

1. Funciones de la capa de lógica del negocio
2. Exponiendo la lógica del negocio
3. Escalando la lógica del negocio
4. Conclusiones

Modelo Three-Tier

- ~~Presentación: interactúa con el usuario para mostrar información y capturar entradas.~~
- Lógica del negocio (o capa de aplicación o capa intermedia): procesa datos, implementa reglas de negocio, realiza cálculos y coordina operaciones.
- ~~Datos: responsable de almacenar, recuperar y gestionar los datos de la aplicación. Incluye las base de datos, sistemas de archivos y otros almacenamientos persistentes.~~

> **Important**
> Las flechas son invocaciones: llamadas a funciones locales, peticiones HTTP, mensajes asíncronos, u otros tipos de comunicación.



Funciones de la capa de lógica del negocio

Es la que se encarga de procesar la información y aplicar las reglas de negocio de la aplicación. Actúa como intermediaria entre la capa de presentación y la capa de datos.

> no debería depender de detalles concretos de HTTP, HTML o una base de datos específica.

- Validación de datos
- Procesamiento de datos: cálculos, transformaciones y manipulaciones
- Implementación de reglas de negocio: calcular precios, aplicar descuentos
- Orquestación de servicios: interacción con otras capas (presentación y datos) y posiblemente con otros servicios externos
- Gestión de la seguridad y autorización
- Manejo de transacciones

> **Tip**
> Conviene mantener los servidores sin estado de sesión en memoria entre peticiones. El estado persistente debe almacenarse en una base de datos o cache.

Funciones de la capa de lógica del negocio

Es la que se encarga de procesar la información y aplicar las reglas de negocio de la aplicación. Actúa como intermediaria entre la capa de presentación y la capa de datos.

> no debería depender de detalles concretos de HTTP, HTML o una base de datos específica.

- Validación de datos
- Procesamiento de datos: cálculos, transformaciones y manipulaciones
- Implementación de reglas de negocio: calcular precios, aplicar descuentos
- Orquestación de servicios: interacción con otras capas (presentación y datos) y posiblemente con otros servicios externos
- Gestión de la seguridad y autorización
- Manejo de transacciones

> **Tip**
> Conviene mantener los servidores sin estado de sesión en memoria entre peticiones. El estado persistente debe almacenarse en una base de datos o cache.

Características Clave

- Independiente de la UI y base de datos
- Reutilizable en diferentes interfaces
- Centraliza las reglas del negocio
- Facilita testing y mantenimiento
- **Expone** la lógica del negocio a través de APIs o servicios API

¿Cómo se expone la lógica del negocio? API

Tipo	Tecnologías	Ejemplo
Funciones	Librerías, SDKs	`pkg.CalcularImpuesto()`
REST	HTTP, JSON, OpenAPI	`GET /api/clientes`
Web Services	SOAP, XML, WSDL	`<soap:Envelope>`
GraphQL	Lenguaje de consulta, Esquema	`query { clientes }`
Event-Driven	Mensajería, Pub/Sub	`nc.SubscribeSync("greet.*")`

¿Cómo se expone la lógica del negocio? API

Tipo	Tecnologías	Ejemplo
Funciones	Librerías, SDKs	`pkg.CalcularImpuesto()`
REST	HTTP,	`GET /api/clientes`
Web Services	SOAP,	`<soap:Envelope>`
GraphQL	Lenguaje de consulta, Esquema	`query { clientes }`
Event-Driven	Mensajería, Pub/Sub	`nc.SubscribeSync("greet.*")`

Por defecto y **sin hacer nada extra**, la lógica del negocio se construye con funciones/métodos.

¿Cómo se expone la lógica del negocio? API

Tipo	Tecnologías	Ejemplo
Funciones	Librerías, SDKs	`pkg.CalcularImpuesto()`
REST	HTTP,	`GET /api/clientes`
Web Services	SOAP,	`<soap:Envelope>`
GraphQL	Lenguaje de consulta, Esquema	`query { clientes }`
Event-Driven	Mensajería, Pub/Sub	`nc.SubscribeSync("greet.*")`

Por defecto y **sin hacer nada extra**, la lógica del negocio se construye con funciones/métodos.

> **Caution**
>
> Las funciones locales son simples y eficientes dentro del proceso. Si otros procesos necesitan invocarlas hay que **diseñar un protocolo, un contrato y mecanismos de comunicación**.

¿De qué depende la elección?

1. ¿Quién va a consumir la lógica del negocio?
   - Aplicaciones web, móviles, otros servicios?
   - Solo la capa de presentación?
   - Usa algún framework o biblioteca específica que requiera un tipo de API?
2. ¿Se espera mucho o poco procesamiento? ¿Cuántos usuarios concurrentes?
3. ¿Es posible definir contratos de servicios con los datos a devolver? ¿Qué tan flexible o cambiante es la interfaz?

¿De qué depende la elección?

1. ¿Quién va a consumir la lógica del negocio?
   - Aplicaciones web, móviles, otros servicios?
   - Solo la capa de presentación?
   - Usa algún framework o biblioteca específica que requiera un tipo de API?
2. ¿Se espera mucho o poco procesamiento? ¿Cuántos usuarios concurrentes?
3. ¿Es posible definir contratos de servicios con los datos a devolver? ¿Qué tan flexible o cambiante es la interfaz?

> **Caution**
> Cualquier cosa que no sean funciones implica **trabajo y consecuencias futuras**.

Funciones en la capa de lógica del negocio

```go
func ValidateEmail(email string) error {
    re := regexp.MustCompile(
    `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`)

    if !re.MatchString(email) {
        return errors.New("formato de email inválido")
    }

    return nil
}

type UserRepository interface {
    Create(email string) (*User, error)
}

func CreateUser(repo UserRepository, email string) (*User, error) {
    if err := ValidateEmail(email); err != nil {
        return nil, fmt.Errorf("error al validar email: %w", err)
    }

    return repo.Create(email)
}
```

Funciones en la capa de lógica del negocio

```go
const minimumOrderAmount = 100

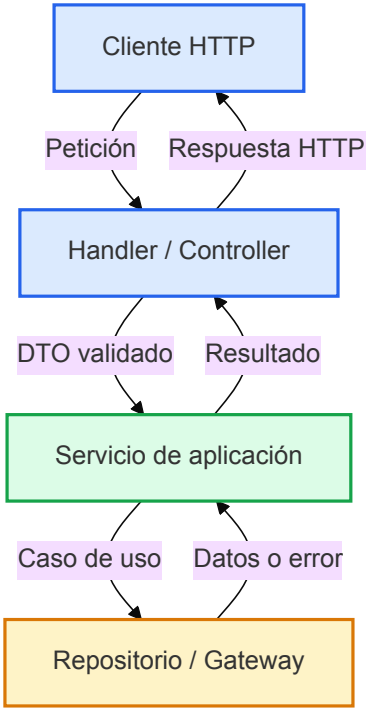
func ProcessOrder(order *Order) error {
    if order.Total < minimumOrderAmount {
        return errors.New("el monto mínimo del pedido es de " +
      fmt.Sprintf("%.2f", minimumOrderAmount))
    }

    fmt.Println("Pedido procesado exitosamente:", order)

    return nil
}
```

Exponiendo la lógica del negocio

Separación de responsabilidades



- **Handler**: interpreta HTTP, valida la entrada básica y construye la respuesta.
- **Servicio**: coordina casos de uso y aplica las reglas del negocio.
- **Repositorio**: encapsula la persistencia o la llamada a otro sistema.
- La lógica del negocio puede probarse sin levantar un servidor HTTP ni una base de datos real.

Validación, reglas y errores

- **Validación de entrada**
  - formato, tipos y campos obligatorios;
  - normalmente en el handler o en un DTO.
- **Reglas del negocio**
  - invariantes y decisiones del dominio;
  - deben vivir en el servicio o dominio.
- **Errores de infraestructura**
  - base de datos, red o servicios externos;
  - se registran y se traducen en un límite de la aplicación.

Validación, reglas y errores

```go
var ErrInsufficientStock = errors.New("stock insuficiente")

func (s *OrderService) Create(ctx context.Context, order Order) error {
    if err := order.Validate(); err != nil {
        return fmt.Errorf("validar pedido: %w", err)
    }
    if err := s.stock.Reserve(ctx, order.Items); err != nil {
        return fmt.Errorf("reservar stock: %w", err)
    }
    return s.orders.Save(ctx, order)
}
```

El handler decide si un error se responde como `400`, `404`, `409` o `500`.

Contexto y cancelación

`context.Context` permite transportar cancelación, deadlines y valores asociados a una petición.

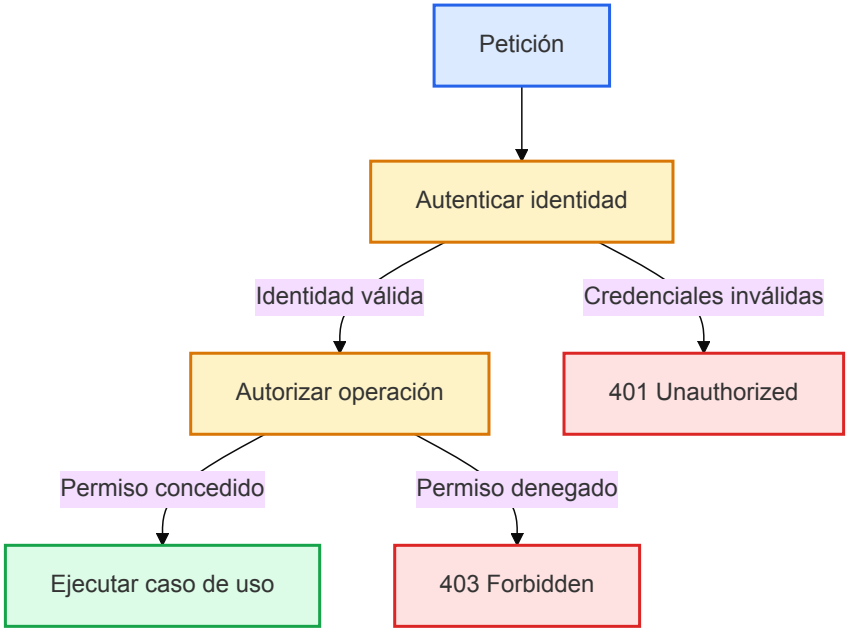
```go
func (s *UserService) Find(ctx context.Context, id int64) (User, error) {
    return s.users.FindByID(ctx, id)
}

func (r *UserRepository) FindByID(ctx context.Context, id int64) (User, error) {
    row := r.db.QueryRowContext(ctx,
        "SELECT id, email FROM users WHERE id = $1", id)
    return scanUser(row)
}
```

Si el cliente cancela la petición o vence el plazo, la operación puede detenerse y liberar recursos.

Autenticación y autorización

- **Autenticación**: determina quién realiza la petición.
  - sesión, token o certificado;
  - produce una identidad verificable.
- **Autorización**: determina qué puede hacer esa identidad.
  - roles, permisos o políticas;
  - debe evaluarse cerca del caso de uso.



La interfaz no es un límite de seguridad: la autorización debe aplicarse también cuando existen otros clientes.

Observabilidad

- **Logs**: registran eventos y errores con contexto estructurado.
- **Métricas**: muestran tendencias y permiten definir alertas.
  - tasa de peticiones;
  - latencia;
  - errores;
  - saturación.
- **Trazas**: siguen una petición entre handlers, servicios y otros sistemas.

```
logger.InfoContext(ctx, "pedido creado",
    "order_id", order.ID,
    "user_id", userID,
)
```

Usar un identificador de correlación ayuda a relacionar logs y trazas de una misma petición.

Salud y apagado ordenado

- **Liveness**: el proceso sigue ejecutándose.
- **Readiness**: la instancia puede recibir tráfico.
- Un fallo de una dependencia puede dejar una instancia no preparada sin reiniciar todo el proceso.

```go
server := &http.Server{Addr: ":8080", Handler: routes}

go server.ListenAndServe()

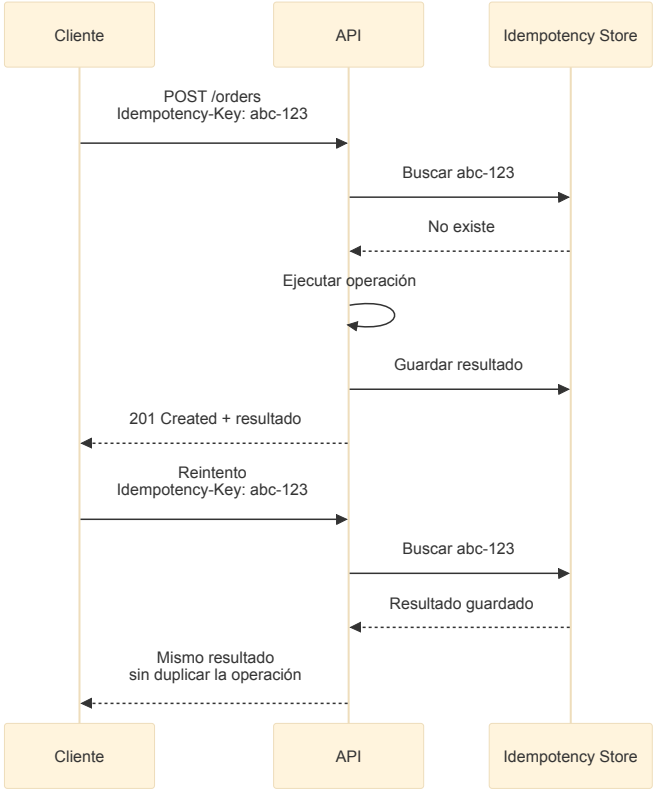
<-signalContext.Done()
shutdownCtx, cancel := context.WithTimeout(
    context.Background(), 10*time.Second)
defer cancel()
server.Shutdown(shutdownCtx)
```

> **Note**
> Al recibir una señal, el servidor deja de aceptar peticiones nuevas, espera a que terminen las activas y fuerza el cierre al vencer el timeout.

El apagado ordenado permite terminar peticiones activas y liberar conexiones.

Idempotencia y asincronía 🔁

- Un cliente o broker puede reintentar una operación.
- Los reintentos pueden producir duplicados.
- Una operación idempotente deja el mismo resultado al ejecutarse varias veces.
- En HTTP puede utilizarse una `Idempotency-Key`.



En consumidores de eventos, usar claves de deduplicación y operaciones idempotentes.

Sincrónico y asíncrono ⇄

Comunicación síncrona: el cliente espera el resultado del caso de uso.

```
Cliente --> API --> Servicio --> Respuesta
```

Procesamiento asíncrono: la API acepta el trabajo y otro componente lo procesa después.

```
Cliente --> API --> Broker --> Consumidor
             |
             +--> 202 Accepted
```

Una respuesta `202 Accepted` confirma la aceptación del trabajo, no su finalización. Debe definirse cómo consultar el resultado y cómo tratar fallos y reintentos.

Exponiendo la lógica del negocio

Motivaciones

- La lógica del negocio se implementa como funciones/métodos
- El problema es que **no es accesible desde otros procesos**
- Tecnologías para presentación Web (, ,) o UI móviles requieren acceder a la lógica del negocio:
  - con determinados protocolos **://**
  - a través de la red

Exponiendo Servicios REST

REST es un estilo para diseñar interfaces que permiten a diferentes aplicaciones **comunicarse a través de HTTP**.

REST (Representational State Transfer) es un estilo arquitectónico para diseñar sistemas distribuidos, basado en:

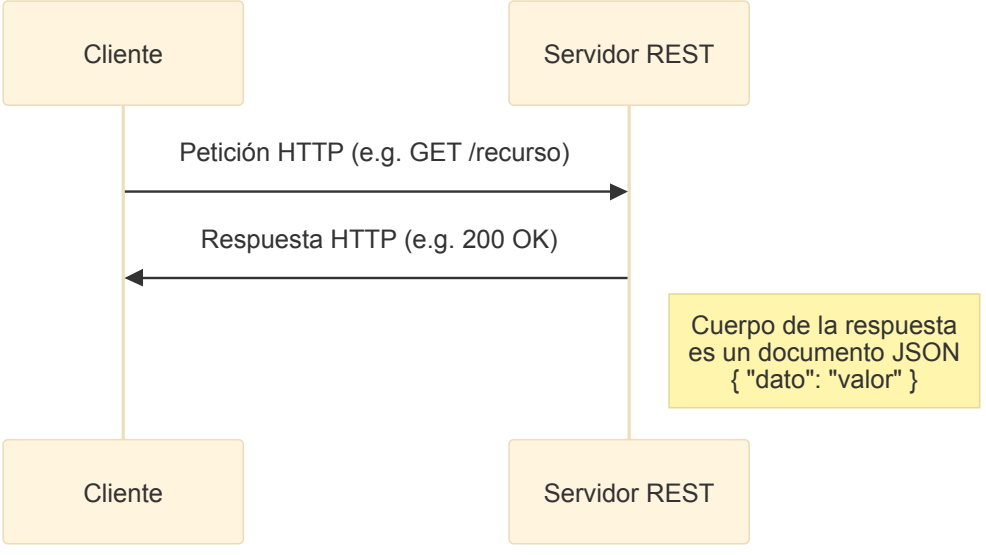
- **Recursos**: muchos conceptos se modelan como recursos identificables por una URL (ej: `/clientes/123`)
- **Operaciones estándar**: Usa verbos HTTP (GET, POST, PUT, DELETE)
- **Sin estado**: Cada petición contiene la información necesaria para procesarse

  > La sesión puede persistir en un sistema externo; el servidor no depende de su memoria local.

- **Representaciones**: JSON es habitual, pero no obligatorio {..}

También importan los códigos de estado, la idempotencia, la cacheabilidad, la autenticación y la evolución del contrato.

Arquitectura de un Servicio REST



- El **cliente** (backend, navegador, aplicación móvil, etc.) envía una petición HTTP a un *endpoint* específico.
- El **servidor REST** procesa la petición y construye una respuesta.
- La respuesta se envía de vuelta al cliente, comúnmente en formato **JSON**.

¿Qué es JSON (JavaScript Object Notation)? {...}

JSON es un formato ligero de intercambio de datos con las siguientes características:

1. Estructura clave-valor (anidada)

```
{
  "name": "John",
  "age": 30,
  "married": true,
  "divorced": false,
  "children": ["Ann", "Billy"],
  "pets": null,
  "cars": [
    { "model": "BMW 230", "mpg": 27.5 },
    { "model": "Ford Edge", "mpg": 24.1 }
  ]
}
```

2. Tipos de datos soportados:

- Strings: `"texto"`
- Números: `123` o `12.34`
- Booleanos: `true` o `false`
- Arrays: `[1, 2, 3]`
- Objetos: `{"clave": "valor"}`

Servidor REST básico con JSON {...}

```go
type Product struct {
    ID    int     `json:"id"`
    Name  string  `json:"name"`
    Price float64 `json:"price"`
}

var products = []Product{
    {1, "Laptop", 999.99},
    {2, "Smartphone", 499.99},
    {3, "Tablet", 299.99},
}

// Ejemplo didáctico: el estado global debe protegerse ante concurrencia
// y reemplazarse por persistencia en una aplicación real.

func main() {
    // Configurar rutas
    http.HandleFunc("/products", productsHandler)
    http.HandleFunc("/products/", productHandler)

    // Iniciar servidor
    log.Println("Server starting on :8080...")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Servidor REST básico con JSON {...}

```go
// Manejador para /products
func productsHandler(w http.ResponseWriter, r *http.Request) {
    switch r.Method {
    case http.MethodGet:
        getProducts(w, r)
    case http.MethodPost:
        createProduct(w, r)
    default:
        http.Error(w, "Method not allowed",
      http.StatusMethodNotAllowed)
    }
}

// Manejador para /products/{id}
func productHandler(w http.ResponseWriter, r *http.Request) {
    // Extraer ID del path
    parts := strings.Split(r.URL.Path, "/")
    if len(parts) != 3 {
        http.Error(w, "Invalid URL", http.StatusBadRequest)
        return
    }
    id, err := strconv.Atoi(parts[2])
```

Servidor REST básico con JSON {...}

```
    if err != nil {
        http.Error(w, "Invalid product ID", http.StatusBadRequest)
        return
    }
    switch r.Method {
    case http.MethodGet:
        getProduct(w, r, id)
    case http.MethodPut:
        updateProduct(w, r, id)
    case http.MethodDelete:
        deleteProduct(w, r, id)
    default:
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
    }
}
// GET /products - Listar todos los productos
func getProducts(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(products)
}
```

Servidor REST básico con JSON {...}

```go
// POST /products - Crear nuevo producto
func createProduct(w http.ResponseWriter, r *http.Request) {
    var newProduct Product

    err := json.NewDecoder(r.Body).Decode(&newProduct)

    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    newProduct.ID = len(products) + 1
    products = append(products, newProduct)
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)

    json.NewEncoder(w).Encode(newProduct)
}
```

Servidor REST básico con JSON {...}

```go
// GET /products/{id} - Obtener producto específico
func getProduct(w http.ResponseWriter, r *http.Request, id int) {

    product, err := findProductByID(id)

    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(product)
}
```

Servidor REST básico con JSON {...}

```go
// PUT /products/{id} - Actualizar producto
func updateProduct(w http.ResponseWriter, r *http.Request, id int) {
    var updatedProduct Product

    err := json.NewDecoder(r.Body).Decode(&updatedProduct)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    index, err := findProductIndexByID(id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    // Mantener el ID original
    updatedProduct.ID = id
    products[index] = updatedProduct
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(updatedProduct)
}
```

Servidor REST básico con JSON {...}

```go
// DELETE /products/{id} - Eliminar producto
func deleteProduct(w http.ResponseWriter, r *http.Request, id int) {
    index, err := findProductIndexByID(id)

    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    products = append(products[:index], products[index+1:]...)
    w.WriteHeader(http.StatusNoContent)
}

// Funciones auxiliares
func findProductByID(id int) (*Product, error) {
    for i := range products {
        if products[i].ID == id {
            return &products[i], nil
        }
    }
    return nil, errors.New("product not found")
}
```

Servidor REST básico con JSON {...}

```go
func findProductIndexByID(id int) (int, error) {
    for i, p := range products {
        if p.ID == id {
            return i, nil
        }
    }
    return -1, errors.New("product not found")
}
```

Probar el servidor:

```bash
# Obtener todos los productos
curl localhost:8080/products
```

```
[
  { "id": 1, "name": "Laptop", "price": 999.99 },
  { "id": 2, "name": "Smartphone", "price": 499.99 },
  { "id": 3, "name": "Tablet", "price": 299.99 }
]
```

```bash
# Obtener el producto #2
curl localhost:8080/products/2
```

```
{ "id": 2, "name": "Smartphone", "price": 499.99 }
```

Servidor REST básico con JSON {…}

```
# Eliminar un producto
curl -i -X DELETE localhost:8080/products/2
```

El servidor responde `HTTP/1.1 204 No Content`: la eliminación fue exitosa y el cuerpo está vacío.

```
# Crear un nuevo producto
curl -X POST http://localhost:8080/products \
     -H "Content-Type: application/json" \
     -d '{"name": "GPS Watch", "price": 199.05}'
```

```
{ "id": 4, "name": "GPS Watch", "price": 199.05 }
```

Alternativa a cURL: Hurl

Hurl es una herramienta que ejecuta peticiones HTTP definidas en un archivo.

Características:

- **Legible y simple**: Define peticiones en archivos de texto plano.
- **Encadenamiento**: Realiza múltiples peticiones en secuencia.
- **Aserciones**: Valida respuestas (códigos de estado, cabeceras, JSON, etc.).
- **Captura de valores**: Extrae datos de una respuesta para usarlos en la siguiente.
- **Integración**: Ideal para testing de APIs y CI/CD.

Alternativa a cURL: Hurl

Un único archivo `requests.hurl` puede contener múltiples peticiones:

```
# requests.hurl

# 1. Obtener todos los productos
GET http://localhost:8080/products
HTTP 200
[Asserts]
jsonpath "$.length()" == 3
jsonpath "$[0].name" == "Laptop"

# 2. Obtener el producto con id 2
GET http://localhost:8080/products/2
HTTP 200
[Asserts]
jsonpath "$.name" == "Smartphone"

# 3. Eliminar el producto con id 2
DELETE http://localhost:8080/products/2
HTTP 204
```

```
# 4. Crear un nuevo producto
POST http://localhost:8080/products
Content-Type: application/json
{
    "name": "GPS Watch",
    "price": 199.05
}
HTTP 201
[Asserts]
jsonpath "$.id" isInteger
jsonpath "$.name" == "GPS Watch"
```

Para ejecutar todas las pruebas:

```
hurl --test requests.hurl
```

Alternativa a cURL: Hurl 🪄

Un único archivo `requests.hurl` puede contener múltiples peticiones:

```
# requests.hurl

# 1. Obtener todos los productos
GET http://localhost:8080/products
HTTP 200
[Asserts]
jsonpath "$.length()" == 3
jsonpath "$[0].name" == "Laptop"

# 2. Obtener el producto con id 2
GET http://localhost:8080/products/2
HTTP 200
[Asserts]
jsonpath "$.name" == "Smartphone"

# 3. Eliminar el producto con id 2
DELETE http://localhost:8080/products/2
HTTP 204
```

```
# 4. Crear un nuevo producto
POST http://localhost:8080/products
Content-Type: application/json
```

⚠ Para pruebas HTTP simples y reproducibles, cURL o Hurl suelen ser suficientes y fáciles de integrar en CI. Herramientas como Postman pueden ser convenientes cuando se necesitan flujos más complejos.

ebas:

```
hurl --test requests.hurl
```

Alternativa a cURL: Hurl

Asserts en Hurl

- Los asserts permiten validar la respuesta de una petición HTTP o su desempeño.
- Se colocan después de la petición, usando palabras clave como `status`, `header`, `body`, etc.
- Ejemplo:

```
GET https://api.example.com/users
HTTP/1.1 200
[Asserts]
status == 200
header "content-type" == "application/json"
duration < 1000  # en ms
```

Sintaxis JsonPath en Hurl

- Hurl soporta JsonPath para validar valores dentro de respuestas JSON.
- Se usa la palabra clave `jsonpath` seguida de la expresión y el valor esperado.
- Ejemplo:

```
GET https://api.example.com/users/1
HTTP/1.1 200
[Asserts]
jsonpath "$.id" matches /\d{4}/
```

Observaciones: servidor REST básico

1. Manejo adecuado de métodos HTTP:

- GET para *recuperar* recursos
- POST para *crear* nuevos recursos
- PUT para *actualizar* recursos existentes
- DELETE para *eliminar* recursos

2. Estructura *limpia* (volveremos):

- Handlers separados para diferentes rutas
- Funciones auxiliares para lógica común
- Manejo centralizado de errores

3. Respuestas JSON adecuadas:

- Content-Type correctamente establecido
- Códigos de estado HTTP apropiados:
  - 200 OK para éxito
  - 201 Created para creación
  - 204 No Content para eliminación
  - 400 Bad Request para errores de cliente
  - 404 Not Found para recursos no existentes
  - 405 Method Not Allowed para métodos no soportados

Aún le falta mucho

1. Validación de entradas:

```
if newProduct.Name == "" || newProduct.Price <= 0 {
    http.Error(w, "Invalid product data", http.StatusBadRequest)
    return
}
```

2. Manejo de concurrencia:

```
var mu sync.Mutex
mu.Lock()
defer mu.Unlock()
// Operaciones sobre products
```

3. Logging:

```
log.Printf("%s %s %s", r.Method, r.URL.Path, r.RemoteAddr)
```

Aún le falta mucho

4. Middleware (ejecuta para todos/algunas peticiones):

```go
func loggingMiddleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      log.Printf("Request: %s %s", r.Method, r.URL.Path)
      next.ServeHTTP(w, r)
  })
}
```

5. Estructura de carpetas:

```
/cmd
  /server
    main.go
/pkg
  /handlers # Manejadores de endpoints
    products.go
  /models   # Estructuras de datos
    product.go
/internal
  /middleware # Middleware (autenticación, logging)
    logging.go
```

Aún le falta mucho

6. Nadie usa una API no documentada:
   - Documentación OpenAPI (Swagger)
   - Ejemplo de documentación:

```yaml
openapi: 3.0.0
info:
  title: Product API
  version: 1.0.0
paths:
  /products:
    get:
      summary: List all products
      responses:
        "200":
          description: A list of products
```

XML sobre HTTP y servicios SOAP

¿Cuándo puede ser útil SOAP?

- **Interoperabilidad**: SOAP es un estándar ampliamente adoptado que permite la comunicación entre aplicaciones en diferentes plataformas y lenguajes. </>
- **Contratos estrictos**: SOAP utiliza WSDL (Web Services Description Language) para definir contratos de servicio, lo que permite una integración más formal y estructurada.
- **Seguridad**: WS-Security define mecanismos para autenticación, integridad y confidencialidad; no se aplican automáticamente por usar SOAP .
- **Soporte para WS-* standards**: SOAP es compatible con una variedad de estándares adicionales como WS-ReliableMessaging, WS-AtomicTransaction, etc., que son útiles en entornos empresariales complejos.

XML sobre HTTP: ejemplo simplificado

```
curl -X POST http://localhost:8080/products \
  -H "Content-Type: text/xml" \
  -d '<GetProductsRequest xmlns="http://example.com/product-service"/>'
```

```
<?xml version="1.0" encoding="UTF-8"?>
<GetProductsResponse>
  <Product>
    <ID>1</ID>
    <Name>Laptop</Name>
    <Price>999.99</Price>
  </Product>
  <Product>
    <ID>2</ID>
    <Name>Phone</Name>
    <Price>499.99</Price>
  </Product>
</GetProductsResponse>
```

Este fragmento muestra XML sobre HTTP. Un servicio SOAP completo también requiere `Envelope`, `Body`, namespaces y un contrato WSDL coherente.

El componente principal es el contrato WSDL



```
<?xml version="1.0"?>
<definitions name="ProductService"
   targetNamespace="http://example.com/product-service"
   xmlns="http://schemas.xmlsoap.org/wsdl/"
   xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
   xmlns:tns="http://example.com/product-service"
   xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <types>
    <xsd:schema targetNamespace="http://example.com/product-service">
      <xsd:element name="GetProductsRequest">
        <xsd:complexType/>
      </xsd:element>
      <xsd:element name="GetProductsResponse">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="Product" maxOccurs="unbounded">
              <xsd:complexType>
                <xsd:sequence>
                  <xsd:element name="ID" type="xsd:int"/>
                  <xsd:element name="Name" type="xsd:string"/>
                  <xsd:element name="Price" type="xsd:decimal"/>
      ....
```

El contrato se implementa en cualquier lenguaje

```go
type Product struct {
    XMLName xml.Name `xml:"Product"`
    ID      int      `xml:"ID"`
    Name    string   `xml:"Name"`
    Price   float64  `xml:"Price"`
}
type GetProductsResponse struct {
    XMLName  xml.Name  `xml:"GetProductsResponse"`
    Products []Product `xml:"Product"`
}

func soapHandler(w http.ResponseWriter, r *http.Request) {
    // Validar método POST
    if r.Method != "POST" {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }
    // Simular datos de productos
    products := []Product{
        {ID: 1, Name: "Laptop", Price: 999.99},
        {ID: 2, Name: "Phone", Price: 499.99},
    }
```

El contrato se implementa en cualquier lenguaje

```
    // Crear respuesta SOAP
    response := GetProductsResponse{Products: products}
    xmlResponse, err := xml.MarshalIndent(response, "", "  ")
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    // Configurar headers y enviar respuesta
    w.Header().Set("Content-Type", "text/xml; charset=utf-8")
    w.WriteHeader(http.StatusOK)
    fmt.Fprint(w, xml.Header)
    w.Write(xmlResponse)
}

func main() {
    http.HandleFunc("/products", soapHandler)
    fmt.Println("SOAP Service listening on :8080...")
    http.ListenAndServe(":8080", nil)
}
```

Exponiendo una API de datos con GraphQL

GraphQL es un lenguaje de consulta y un entorno de ejecución para cumplir con esas consultas:

- **Consulta Específica**: Los clientes solicitan exactamente los datos que necesitan, evitando *over-fetching* o *under-fetching* de datos.
- **Endpoint concentrado**: Muchas APIs GraphQL usan un único punto de entrada, aunque es una convención y no una obligación.
- **Esquema Definido**: La API se define mediante un esquema que describe los tipos de datos disponibles y las operaciones que se pueden realizar.
- **Fuertemente Tipado**: El esquema proporciona un contrato claro entre el cliente y el servidor.

Exponiendo una API de datos con GraphQL

- Aplicaciones complejas con muchas dependencias de datos:
  - Permite a los clientes obtener solo los datos necesarios en una sola petición, mejorando el rendimiento.
- Aplicaciones móviles con ancho de banda limitado:
  - Reducir la cantidad de datos transferidos es crucial.
- Frontends que necesitan flexibilidad en la obtención de datos:
  - Los desarrolladores de frontend tienen más control sobre qué datos reciben.
- Evolución rápida de la API:
  - El esquema bien definido facilita la adición de nuevos campos sin romper las consultas existentes.

Exponiendo una API de datos con GraphQL

- Aplicaciones complejas con muchas dependencias de datos: 🗄️
  - Permite a los clientes obtener solo los datos necesarios en una sola petición, mejorando el rendimiento.
- Aplicaciones móviles con ancho de ba
  - Reducir la cantidad de datos transf
- Frontends que necesitan flexibilidad en
  - Los desarrolladores de frontend tien s reciben.
- Evolución rápida de la API: ⚡
  - El esquema bien definido facilita la adición de nuevos campos sin romper las consultas existentes.

Es *similar* a exponer un método para ejecutar un `SELECT` en una base de datos, con la flexibilidad de elegir qué campos y relaciones obtener.

Escalando la lógica del negocio



Contenidos

1. Escalando la lógica del negocio
2. Motivaciones
3. Escalando con un Load Balancer
4. Nginx como Load Balancer
5. Escalando con Docker Swarm
6. Servicio escalable con Docker Swarm
7. Los límites del monolito
8. Dividir la lógica del negocio en varios servicios
9. Dividir la lógica del negocio en microservicios
10. Aspectos operativos adicionales

Motivaciones

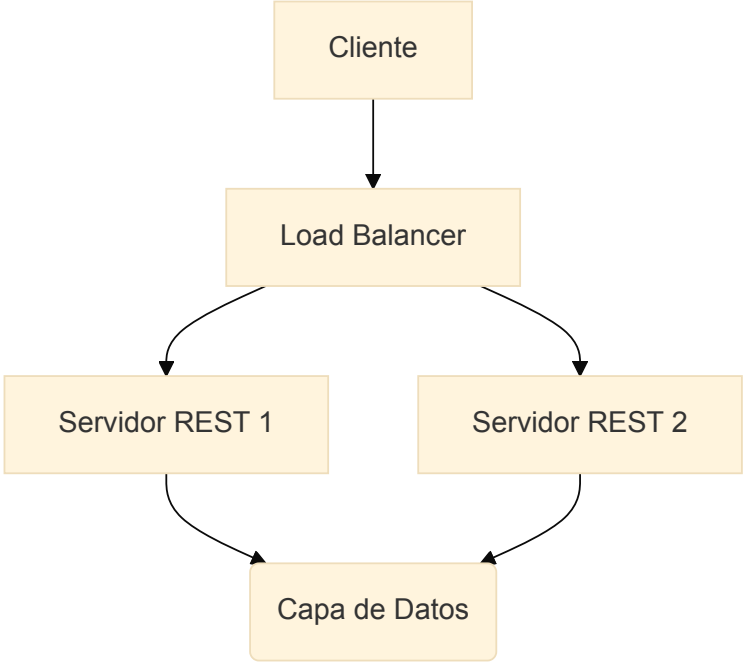
Las soluciones previas son **monolíticas**: *toda la lógica del negocio se ejecuta en el mismo proceso*.

Motivaciones

Las soluciones previas son **monolíticas**: *toda la lógica del negocio se ejecuta en el mismo proceso*.

> **Caution**
>
> Puede limitar el escalado selectivo y el aislamiento de fallos, aunque un monolito también puede escalar horizontalmente.
>
> El problema no es la concurrencia por sí misma, sino que toda la aplicación suele desplegarse y escalarse como una unidad.

Escalando con un Load Balancer



Utilizar un Load Balancer para distribuir las peticiones entre múltiples instancias de la lógica del negocio, permitiendo que cada instancia maneje una parte de la carga:

- 👍 Requisitos mínimos.
- 👍 Muy fácil y liviano.
- 👎 Una implementación simple puede ser un punto único de falla: agregar redundancia.
- 👎 Despliegue manual: no dinámico/adaptable.

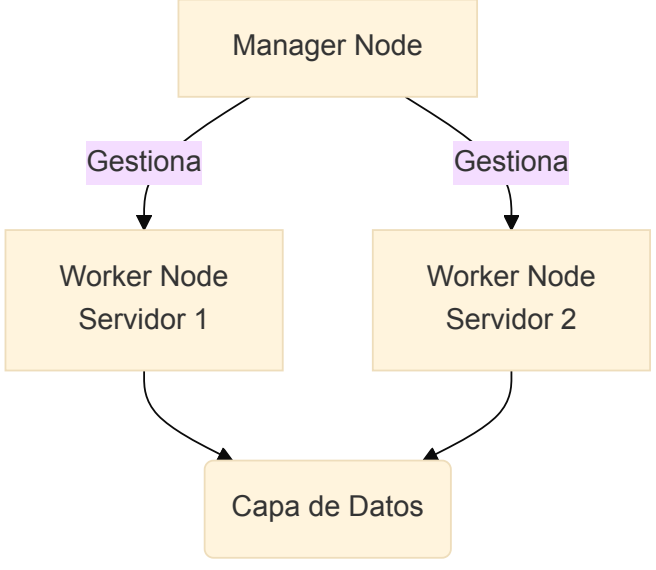
Nginx como Load Balancer

```
http {
    upstream backend {
        server backend1.example.com weight=3;
        server backend2.example.com;
        server backend3.example.com;
        server 192.0.0.1 backup;
    }

    server {
        listen 80;
        location /api/ {
            proxy_pass http://backend;
        }
    }
}
```

- Nginx distribuye las peticiones entre los servidores según sus pesos relativos.
- En este ejemplo, `backend1` recibe aproximadamente tres veces más peticiones que cada servidor con peso `1`.
- El servidor marcado como `backup` solo recibe tráfico si los servidores principales no están disponibles.

Escalando con Docker Swarm



- Nodos Managers: Gestionan el cluster
- Nodos Workers: Ejecutan las cargas de trabajo
- Servicios: Aplicaciones contenerizadas
- Stacks: Conjuntos de servicios relacionados

Consideraciones:

- 👎 Infraestructura más compleja.
- 👍 Hay un *orquestador* que crea/elimina.
- 👍 Cada servidor en su propio .
- 👍 Actualizaciones sin downtime (rolling updates).

Servicio escalable con Docker Swarm

docker-compose.yml

```yaml
services:
  rest-api:
    image: my-rest-api:latest
    ports:
      - "8080:8080"
    deploy:
      replicas: 6
      update_config:
        parallelism: 2
        delay: 10s
      restart_policy:
        condition: on-failure
    networks:
      - api-net

networks:
  api-net:
    driver: overlay
```

Comando para iniciar el servicio:

```
docker stack deploy -c docker-compose.yml myapp
```

Escalado:

- Manual: `docker service scale`
- El autoscaling basado en CPU o requests requiere herramientas y métricas adicionales.

> **Tip**
>
> Previo a desplegar hay que crear el cluster de Docker Swarm.

Servicio escalable con Docker Swarm

docker-compose.yml

```yaml
services:
  rest-api:
    image: my-rest-api:latest
    ports:
      - "8080:8080"
    deploy:
      replicas: 6
      update_config:
        parallelism: 2
        delay: 10s
      restart_policy:
        condition: on-failure
    networks:
      - api-net

networks:
  api-net:
    driver: overlay
```

Comando para iniciar el servicio:

```
docker stack deploy -c docker-compose.yml myapp
```

Escalado:

```
rvice scale
```

do en CPU o

herramientas y

es.

Siguen siendo varios monolitos. Eventualmente el tamaño de la lógica del negocio puede ser tan grande que se vuelve necesario dividirla.

Tip

Previo a desplegar hay que crear el cluster de Docker Swarm.

Servicio escalable con Docker Swarm

Monitorear servicios:

```
docker service ps myapp_rest-api
```

Escalar manualmente

```
docker service scale myapp_rest-api=10
```

Rolling update (cuando hay una nueva versión de la imagen):

```
docker service update --image my-rest-api:v2 --update-parallelism 2 \
  --update-delay 45s myapp_rest-api
```

Los límites del monolito

1. Acoplamiento Estructural
   - **Problema**: Cambios en un módulo afectan a todo el sistema
   - **Ejemplo**: Actualizar biblioteca de pagos requiere redeploy completo
   - **Impacto**: Alto riesgo en modificaciones, pruebas extensas
2. Escalabilidad Uniforme
   - **Problema**: Escalar un componente = escalar todo
   - **Caso**: Picos en procesamiento de imágenes escalan toda la aplicación
   - **Costo**: Recursos infrautilizados en otros módulos

Los límites del monolito

3. Barreras Tecnológicas
   - **Limitación**: Todo el equipo usa mismo stack tecnológico
   - **Ejemplo**: No se puede usar Python para ML si el monolito es Go
   - **Consecuencia**: Imposibilidad de usar herramienta óptima para cada tarea
4. Puntos Únicos de Falla
   - **Riesgo**: Un fallo puede afectar a todo el sistema si no hay aislamiento
   - **Escenario**: Error en módulo de reportes inaccesibiliza checkout
   - **MTTR**: Tiempos de recuperación prolongados

Dividir la lógica del negocio en varios servicios

Servicio de Recomendaciones (alto CPU):

```
func main() {
    http.HandleFunc("/recommend", recommendHandler)
    log.Fatal(http.ListenAndServe(":8081", nil))
}
```

Servicio de Usuarios (mucha memoria):

```
func main() {
    http.HandleFunc("/users", usersHandler)
    log.Fatal(http.ListenAndServe(":8082", nil))
}
```

Servicios otros:

```
func main() {
    http.HandleFunc("/general", genericHandler)
    log.Fatal(http.ListenAndServe(":8083", nil))
}
```

Dividir la lógica del negocio en varios servicios

Servicio de Recomendaciones (alto CPU):

```
func main() {
    http.HandleFunc("/recommend", recommendHandler)
    log.Fatal(http.ListenAndServe(":8081", nil))
}
```

Servicio de Usuarios (mucha memoria):

```
func main() {
    http.HandleFunc("/users", usersHandler)
    log.Fatal(http.ListenAndServe(":8082", nil))
}
```

Servicios otros:

```
func main() {
    http.HandleFunc("/general", genericHandler)
    log.Fatal(http.ListenAndServe(":8083", nil))
}
```

Caution

- Qué tanto subdividir la lógica del negocio?
- Cómo asigno servicios a servidores?
- Qué pasa si un servicio falla?

Dividir la lógica del negocio en microservicios λ

Dividir una aplicación monolítica en pequeños servicios independientes que se comunican entre sí a través de la red (message bus, REST o gRPC).

- Escalado independiente: cada microservicio puede escalarse según sus necesidades, si no comparte cuellos de botella.
- Despliegue independiente: requiere límites claros y pipelines independientes.
- Tecnologías diversas: es posible elegir tecnologías distintas, a costa de mayor complejidad operativa.
- Mayor resiliencia: puede aislar fallos si se incorporan timeouts, reintentos y degradación controlada.

> **Tip**
>
> El orquestador automatiza parte de la infraestructura. El equipo debe definir recursos, réplicas, health checks, políticas de despliegue, seguridad y observabilidad.

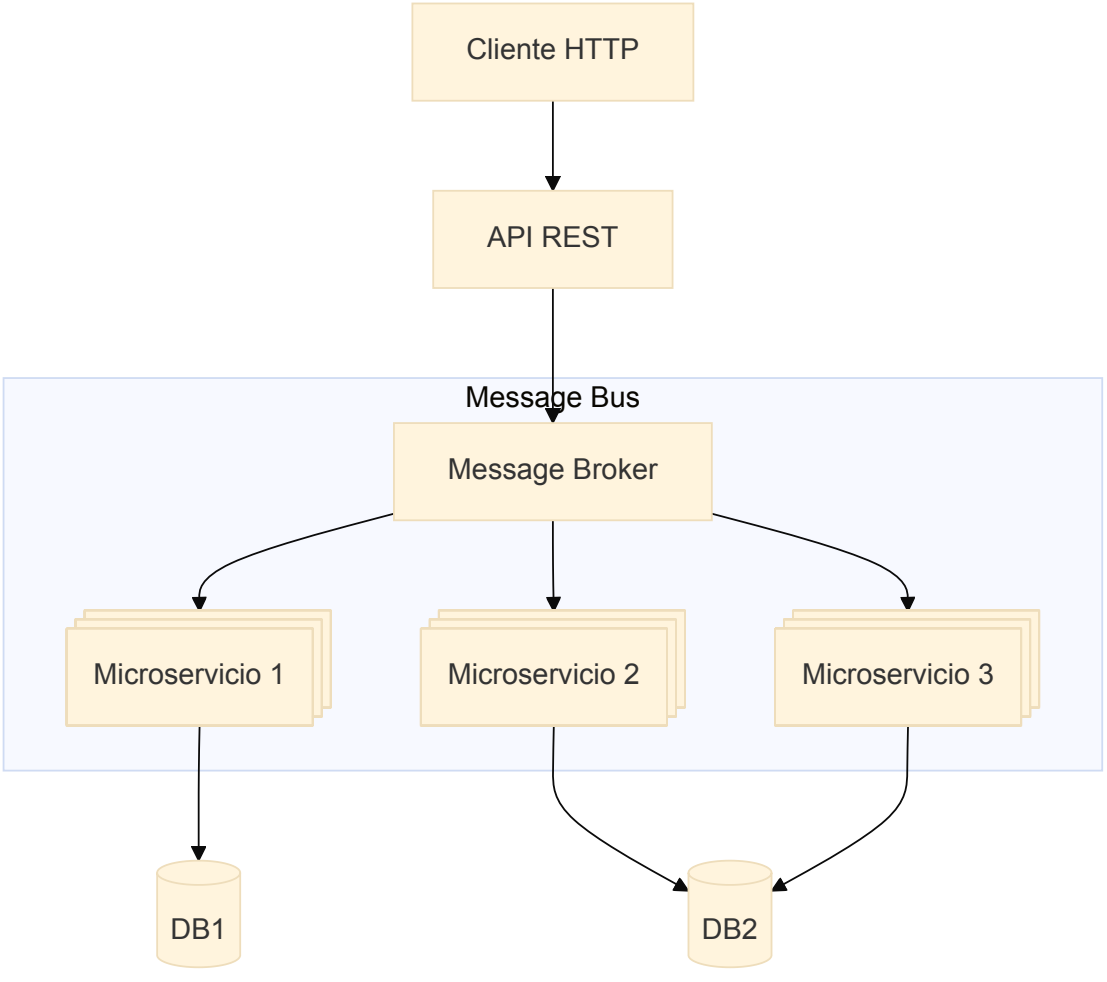
Dividir la lógica del negocio en microservicios λ

Dividir una aplicación monolítica en pequeños servicios independientes que se comunican entre sí a través de la red (message bus, REST o gRPC).

- Escalado independiente: cada microservicio puede escalarse según sus necesidades, si no comparte cuellos de botella.
- Despliegue independiente: requiere límites claros y pipelines independientes.
- Tecnologías diversas: es posible elegir tecnologías distintas, a costa de mayor complejidad operativa.
- Mayor resiliencia: puede aislar fallos si se incorporan timeouts, reintentos y degradación controlada.

> **Tip**
>
> El orquestador automatiza parte de la infraestructura. El equipo debe definir recursos, réplicas, health checks, políticas de despliegue, seguridad y observabilidad.

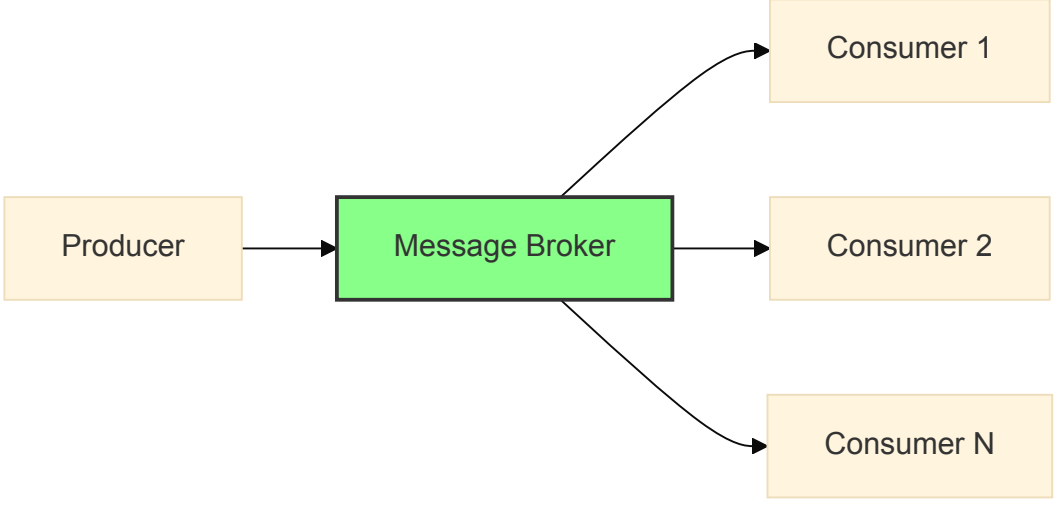
Microservicios con un message broker λ



Microservicios con un message broker

Un message broker es un middleware que:

- **Recibe** mensajes de aplicaciones *productoras* (publishers)
- **Enruta** estos *mensajes* según reglas predefinidas
- **Entrega** los mensajes a aplicaciones *consumidoras* (subscribers)



Funciones de un message broker

- **Transformación de mensajes**: Puede transformar mensajes cuando se configura para ello, permitiendo la comunicación entre sistemas con diferentes formatos.
- **Enrutamiento de mensajes**: Puede recibir un mensaje y distribuirlo entre uno o más destinatarios según sea necesario.
- **Colas de mensajes**: Gestiona colas y puede ofrecer distintas garantías de entrega. El orden y la durabilidad dependen de la configuración.
- **Filtrado de mensajes**: Puede filtrar mensajes según ciertos criterios, como el origen del mensaje o el contenido específico.
- **Soporte para diversos protocolos de comunicación**: Puede trabajar con protocolos como AMQP o MQTT.

La entrega puede ser *at-most-once* o *at-least-once*. Los consumidores deben contemplar reintentos, duplicados e idempotencia.

REST con microservicios en un message broker

Beneficios de REST:

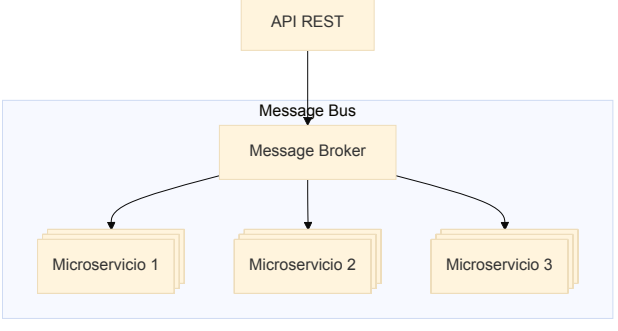
- Facilidad de consumo
- Estándar ampliamente adoptado
- Simplicidad para CRUD

Ventajas (veremos NATS):

- Escalado horizontal mediante múltiples consumidores y configuración adecuada
- Patrones avanzados (Pub/Sub, Queue Groups)
- Baja latencia en condiciones favorables; depende de la red y la configuración
- Tolerancia a fallos

Caso de uso ideal:

- Actualizaciones en tiempo real
- Procesamiento asíncrono
- Alta escalabilidad
- Integración con múltiples sistemas



Servicio REST principal

```
import (
  ...

    "github.com/nats-io/nats.go"
)

type Product struct {
    ID    string  `json:"id"`
    Name  string  `json:"name"`
    Price float64 `json:"price"`
}

var nc *nats.Conn

func main() {
    // Conectar a NATS
    var err error
    nc, err = nats.Connect("nats://localhost:4222")
    if err != nil {
        log.Fatal(err)
    }
    defer nc.Close()
```

```
    // Configurar HTTP server
    http.HandleFunc("/products", productsHandler)
    log.Println("Server running on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

func productsHandler(w http.ResponseWriter,
    r *http.Request) {
    switch r.Method {
    case http.MethodGet:
        getProducts(w, r)
    case http.MethodPost:
        createProduct(w, r)
    default:
        http.Error(w, "Method not allowed",
        http.StatusMethodNotAllowed)
    }
}
```

Servicio REST principal

```go
func createProduct(w http.ResponseWriter, r *http.Request) {
    var p Product
    if err := json.NewDecoder(r.Body).Decode(&p); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    // Publicar evento de creación
    type ProductCreated struct {
        Type    string    `json:"type"`
        Product Product   `json:"product"`
        Time    time.Time `json:"time"`
    }

    event := ProductCreated{Type: "product_created", Product: p, Time: time.Now()}
    eventData, err := json.Marshal(event)
    if err != nil {
        http.Error(w, "Error creating event", http.StatusInternalServerError)
        return
    }
    if err := nc.Publish("products.events", eventData); err != nil {
        http.Error(w, "Error processing request", http.StatusInternalServerError)
        return
    }
```

Servicio REST principal

```
    // Responder al cliente
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusAccepted)
    json.NewEncoder(w).Encode(map[string]string{
        "status":  "processing",
        "message": "Product creation in progress",
    })
}
```

Consumidor de Eventos

```
import (
  ...
  "github.com/nats-io/nats.go"
)

func main() {
    nc, err := nats.Connect("nats://localhost:4222")
    if err != nil {
        log.Fatal(err)
    }
    defer nc.Close()

    // Suscribirse a eventos
    _, err = nc.Subscribe("products.events", func(msg *nats.Msg) {
        var event struct {
            Type    string  `json:"type"`
            Product Product `json:"product"`
        }
        err := json.Unmarshal(msg.Data, &event)
        if err != nil {
            log.Printf("Error decoding event: %v", err)
            return
        }
```

Consumidor de Eventos

```
        switch event.Type {
        case "product_created":
            log.Printf("Processing new product: %s", event.Product.Name)
            // Lógica de negocio aquí...
            time.Sleep(1 * time.Second) // Simular procesamiento
            log.Printf("Product %s processed", event.Product.Name)
        }
    })

    if err != nil {
        log.Fatal(err)
    }

    log.Println("Event consumer running...")
    select {} // Mantener el programa en ejecución
}
```

Conclusiones: message broker

- Escalado horizontal: se pueden añadir consumidores; el autoscaling requiere métricas y mecanismos adicionales
- Resiliencia: Si un servicio falla, los mensajes se mantienen y se atenderán más tarde
- Observabilidad: puede facilitarse con persistencia, retención y trazabilidad configuradas
- Procesamiento asíncrono: Operaciones largas no bloquean la API
- Heterogeneidad: diferentes lenguajes y tecnologías para cada microservicio

- Complejidad adicional: Requiere más infraestructura y monitoreo
- Programación más compleja: Manejo de eventos y asincronía
- La comunicación entre servicios puede ser más difícil de depurar y más cara en términos de latencia

Aspectos operativos adicionales

- **Contratos**: versionar APIs y eventos manteniendo compatibilidad hacia atrás.
- **Resiliencia**: usar timeouts, reintentos con backoff y circuit breakers al llamar otros servicios.
- **Datos**: definir el ownership de cada dato y contemplar consistencia eventual.
- **Carga**: aplicar backpressure para evitar saturación de consumidores y dependencias.
- **Pruebas**: combinar pruebas unitarias, de integración y de contrato.

Estos mecanismos agregan complejidad: deben incorporarse cuando el problema lo justifica, no por defecto.

Conclusiones

- El monolito es fácil de desarrollar y desplegar, pero **limita la escalabilidad y la flexibilidad**
- Dependiendo de cómo esté estructurado el monolito, puede ser difícil de mantener y escalar... *Volveremos con esto...*
- No caer en la trampa de pensar que un monolito es "suficiente" para siempre
- Una capa de lógica del negocio escalable y distribuida debe ser **planificada cuidadosamente**
- Hay arquitecturas como microservicios y event-driven que permiten una *mayor flexibilidad y escalabilidad, pero incrementan la complejidad*
- No hay una solución única para todos los casos, cada enfoque tiene sus pros y contras